

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №7

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: " Обход графа в глубину"

Выполнил:

Студент группы 24ВВВ3

Макунькин А.М.

Мелюшев М.М.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение обхода графов в глубину.

Теоретическая часть

Обход в глубину — это фундаментальный алгоритм, используемый для систематического посещения всех вершин графа.

Его основная идея заключается в том, чтобы, начав с выбранной вершины, двигаться вглубь графа, насколько это возможно, прежде чем возвращаться назад (делать "откат" или "backtrack").

Ключевые принципы:

Выбор начальной вершины: Алгоритм начинает работу с произвольно выбранной вершины.

Помеченные посещенных вершин: чтобы избежать зацикливания и повторного посещения одной и той же вершины, каждая посещенная вершина помечается специальным образом (например, заносится в массив visited).

Рекурсивное погружение: для текущей вершины алгоритм просматривает всех ее "соседей" (смежные вершины). Как только находится не посещенный сосед, алгоритм рекурсивно применяется к этой новой вершине.

Возврат (Backtrack): когда у текущей вершины не остается не посещенных смежных вершин, алгоритм возвращается на шаг назад (к вершине, из которой пришел) и продолжает обход оттуда.

Этот процесс повторяется до тех пор, пока все вершины, достижимые из начальной, не будут посещены. Если граф несвязный, алгоритм необходимо запустить повторно для не посещенных вершин.

Псевдокод

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОГ

- 1.1. для всех i положим $NUM[i] = False$ пометим как "не посещенную";
- 1.2. ПОКА существует "новая" вершина v
- 1.3. ВЫПОЛНЯТЬ DFS (v).

Алгоритм DFS(v):

- 2.1. пометить v как "посещенную" $NUM[v] = True$;
- 2.2. вывести на экран v ;
- 2.3. ДЛЯ $i = 1$ ДО $size_G$ ВЫПОЛНЯТЬ

```

2.4.  ЕСЛИ  $G(v,i) == 1$  И  $NUM[i] == \text{False}$ 
2.5.      ТО
2.6.      {
2.7.      DFS(i);
2.8.      }

```

Лабораторное задание:

- а. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
- б. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.

Реализация состоит из подготовительной части, в которой все вершины помечаются как не помеченные (п.1.1) и осуществляется запуск процедуры обхода для вершин графа (п.1.2, 1.3). И непосредственно процедуры обхода, которая помечает текущую (т.е. ту, в которой на текущей итерации находится алгоритм) вершину как посещенную (п. 2.1). Затем выводит номер текущей вершины на экран (п.2.2) и в цикле просматривает v -ю строку матрицы смежности графа $G(v,i)$. Как только алгоритм встречает смежную с v не посещенную вершину (п.2.4), то для этой вершины вызывается процедура обхода (п.2.7).

Например, пусть дан граф (рисунок 1), заданный в виде матрицы смежности:

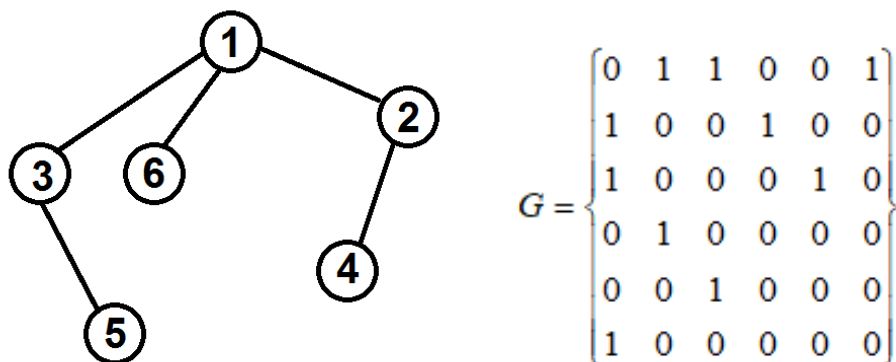
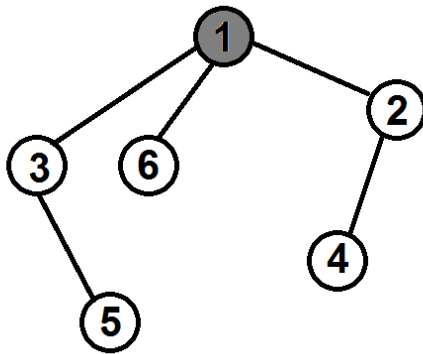


Рисунок 1 – Граф

Тогда, если мы начнем обход из первой вершины, то на шаге 2.1 она

будет помечена как посещенная ($NUM[1] = \text{True}$), на экран будет выведена единица.



$$NUM = \{\text{True} \quad \text{False} \quad \text{False} \quad \text{False} \quad \text{False} \quad \text{False}\}$$

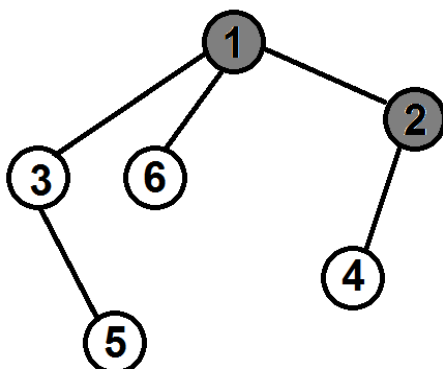
Рисунок 2 – Вызов DFS(1)

При просмотре 1-й строки матрицы смежности

$$G = \begin{pmatrix} 0 & \boxed{1} & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

будет найдена смежная вершина с индексом 2 ($G(1,2) = 1$), которая не посещена ($NUM[2] = \text{False}$) и будет вызвана процедура обхода уже для нее - DFS(2).

На следующем вызове на шаге 2.1 вершина 2 будет помечена как посещенная ($NUM[2] = \text{True}$), на экран будет выведена двойка.



$$NUM = \{\text{True} \quad \text{True} \quad \text{False} \quad \text{False} \quad \text{False} \quad \text{False}\}$$

Рисунок 3 – Вызов DFS(2)

И алгоритм перейдет к просмотру второй строки матрицы смежности. Первая смежная с вершиной 2 - вершина с индексом 1 ($G(2,1) = 1$),

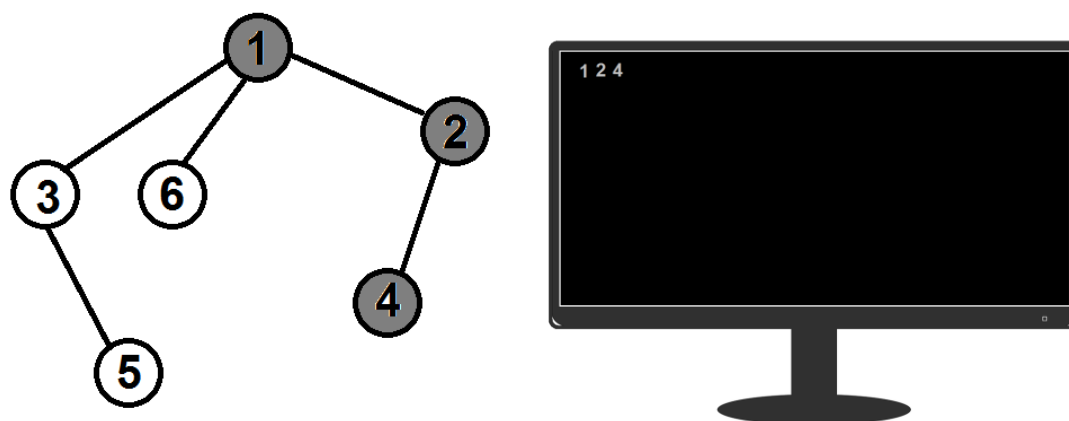
$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

которая к настоящему моменту уже посещена ($NUM[1] = \text{True}$) и процедура обхода для нее вызвана не будет. Цикл 2.3 продолжит просмотр матрицы смежности.

Следующая найденная вершина, смежная со второй, будет иметь индекс 4 ($G(2,4) = 1$), она не посещена ($NUM[4] = \text{False}$) и для нее будет вызвана процедура обхода - DFS(4).

$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

Вершина 4 будет помечена как посещенная ($NUM[4] = \text{True}$), на экран будет выведена четверка.



$$NUM = \{\text{True} \quad \text{True} \quad \text{False} \quad \text{True} \quad \text{False} \quad \text{False}\}$$

Рисунок 4 – Вызов DFS(4)

При просмотре 4-й строки матрицы будет найдена вершина 2, но она уже посещена ($NUM[2] = True$), поэтому процедура обхода вызвана не будет.

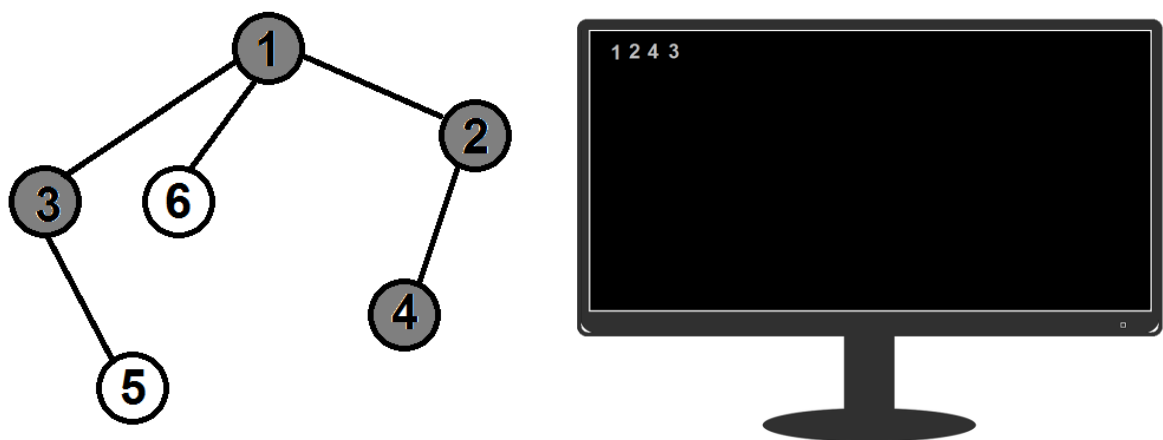
$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

Цикл 2.3 завершится и для текущего вызова $DFS(4)$ процедура закончит свою работу, вернувшись к точке вызова, т.е. к моменту просмотра циклом 2.3 строки с индексом 2 для вызова $DFS(2)$.

В вызове $DFS(2)$ цикл 2.3 продолжит просмотр строки 2 в матрице смежности, и, пройдя её до конца завершится. Вместе с этим завершится и вызов процедуры $DFS(2)$, вернувшись к точке вызова - просмотру циклом 2.3 строки с индексом 1 для вызова $DFS(1)$.

При просмотре строки 1 циклом 2.3 в матрице смежности будет найдена следующая не посещенная, смежная с 1-й, вершина с индексом 3 ($G(1,2) = 1$ и $NUM[3] = False$) и для нее будет вызвана $DFS(3)$.

Вершина 3 будет помечена как посещенная ($NUM[3] = True$), на экран будет выведена тройка.



$$NUM = \{True \quad True \quad True \quad True \quad False \quad False\}$$

Рисунок 4 – Вызов $DFS(3)$

Работа алгоритма будет продолжаться до тех пор, пока будут оставаться

не посещенные вершины, т.е. для которых $NUM[i] == False$.

В конце работы алгоритма все вершины будут посещены. А на экран будут выведены номера вершин в порядке их посещения алгоритмом.



$NUM = \{True \quad True \quad True \quad True \quad True \quad True\}$

Рисунок 5 – Результат работы обхода

Результат программы

```
Консоль отладки Microsoft Vi
Input number of vertission: 8
0 1 1 1 0 1 1 0
1 0 0 0 1 0 1 0
1 0 0 0 1 1 1 1
1 0 0 0 0 1 1 0
0 1 1 0 0 1 0 1
1 0 1 1 1 0 0 1
1 1 1 1 0 0 0 0
0 0 1 0 1 1 0 0
Input the start vert:5
Path: 5 0 1 4 2 6 3 7
D:\pr\Lab7.2L\x64\Debug\Lab7.2L.exe (процесс 83820) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 6 – обход графа

Листинг

Файл 1-Lab3L.c

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>

void DFS(int** G, int numG, int* visited, int s) {
    visited[s] = 1;
    printf("%3d", s);

    for (int i = 0; i < numG; i++) {
        if (G[s][i] == 1 && visited[i] == 0)
            DFS(G, numG, visited, i);
    }
}

int main()
{
    int** G;
    int numG, current;
    int* visited;

    printf("Input number of vertission: ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));

    for (int i = 0; i < numG; i++)
        G[i] = (int*)malloc(numG * sizeof(int));

    srand(time(NULL));

    for (int i = 0; i < numG; i++)
    {
        visited[i] = 0;
        for (int j = i; j < numG; j++)
        {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }
    for (int i = 0; i < numG; i++)
    {
        for (int j = 0; j < numG; j++)
        {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }
    printf("Input the start vert:");
    scanf("%d", &current);

    printf("Path: ");

    DFS(G, numG, visited, current);

    free(visited);
    for (int i = 0; i < numG; i++)
        free(G[i]);
    free(G);
    return 0;
}
```


Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №7. В процессе выполнения работы были использованы знания о обходе графа в глубину.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/LareklAREK/LiOAiIZ-2OURSE-.git>

<https://github.com/Motorrr/LiOAvIZ.git>